CASE STUDY

NOVEMBER 2026

Paying exactly once: moving an AI agent's refund approvals onto Temporal and Nexus

How a support team can let an LLM agent propose refunds, keep a human in the loop for as long as the approval takes, and hand the money movement to the payments team behind a contract, without paying anyone twice.



AUTHOR
Kevin M. Butler

RUNNABLE EVIDENCE
github.com/agileresearchservices/paid-once

STACK
Temporal · Nexus · LangGraph

Contents

	At a glance	3
	Key takeaways	4
01	The situation	5
02	Why it breaks	6
03	Who faces this, and why now	6
04	The options	8
05	The migration: one process, refund approval	9
06	Running the transition	12
07	Results	14
08	Benefits, costs and limitations	17
09	Judging the pilot and deciding whether to expand	18
10	A recommended path	18
11	Lessons for teams building agent actions	19
App. A	Reproduce the results	20
App. B	Notes for Temporal's field, docs and product teams	20
App. C	What my experience supports, and what needs validation	21

About this case study. The company here is representative, not a named customer. It is adapted from the kind of production agent work I have done, with no employer or client specifics, and every line of the supporting repository was written from scratch for it. The results come from running both the existing system and the migrated one, side by side, under the same injected failures, and scoring both from the payments ledger alone. Claims carry a tag so you can tell evidence from argument:

Tag	Meaning
MEASURED	Scored by the ledger audit in a reproducible `make scenario` run
OBSERVED	Seen by inspecting Temporal or the code; not scored
DOCUMENTED	Stated in Temporal's documentation (linked)
EXPECTED	My reasoning; not built or tested
ASSUMPTION	Needs a real customer's numbers to confirm

At a glance

22 of 22
approved refunds paid exactly once on Temporal + Nexus

9–10 of 22
paid more than once by today's stack, same faults

3
approvals never paid by today's stack, even after the key fix

0
errors the caller saw through four Nexus-path failures

Industry	Online marketplace (the pattern applies to any business whose agent can move money)
Process migrated	Refund approval: an LLM agent proposes a refund, a human approves it, payments moves the money
Starting stack	Job queue with at-least-once delivery, worker pool, LangGraph agent with an in-memory checkpointer, HTTPS call to a payments API owned by another team
Teams involved	Support/AI (owns the agent), payments platform (owns the money), finance/risk (owns the audit question)
What changed	The approval wait became a Temporal workflow waiting on a signal; the refund became a Nexus operation the payments team publishes and runs in its own namespace
What stayed	Ticket intake, the agent's decision logic, the LLM provider, the payments API and its ledger
Result	Under worker kills, queue redelivery and a payments service that commits and then stalls, the Temporal path refunded all 22 approved tickets exactly once; the existing stack paid 9 to 10 of them more than once and never paid 3 MEASURED
The surprise	A one-line change to the existing stack's idempotency key removed every duplicate. It did not recover the 3 approvals that died with their worker MEASURED
Recommendation	Make the key fix this week regardless. Move refund approval onto Temporal if approvals wait hours or days; add Nexus when a different team owns the payments API

Key takeaways

1. **The dangerous failure is not a crash; it is not knowing.** Payments committed the refund and the reply never arrived, or a worker died after the money moved. A retry is then either unsafe (a second refund) or impossible (the state is gone).

2. **Durable execution is not exactly-once by itself.** The refund was safe on Temporal because the idempotency key came from the business operation (`refund-<ticket_id>`), not from the attempt. A key of the same kind removed the duplicates from the existing stack too.

3. **What the platform gave that the key fix did not** was the approval wait itself: on Temporal it survives restarts and deploys by default; on the existing stack it lives in worker memory, and fixing that means building a durable checkpointer and a recovery sweeper.

4. **Nexus earned its place at the team boundary.** Four failures in a row (a stalled payments reply, then the support worker, the payments worker and the Temporal server itself) left the support workflow with no error and every refund paid once. Payments absorbed its own failures with retries the caller never saw OBSERVED.

5. **Prove it from the ledger.** Score old and new from the system of record, never from either system's own state.

01 The situation

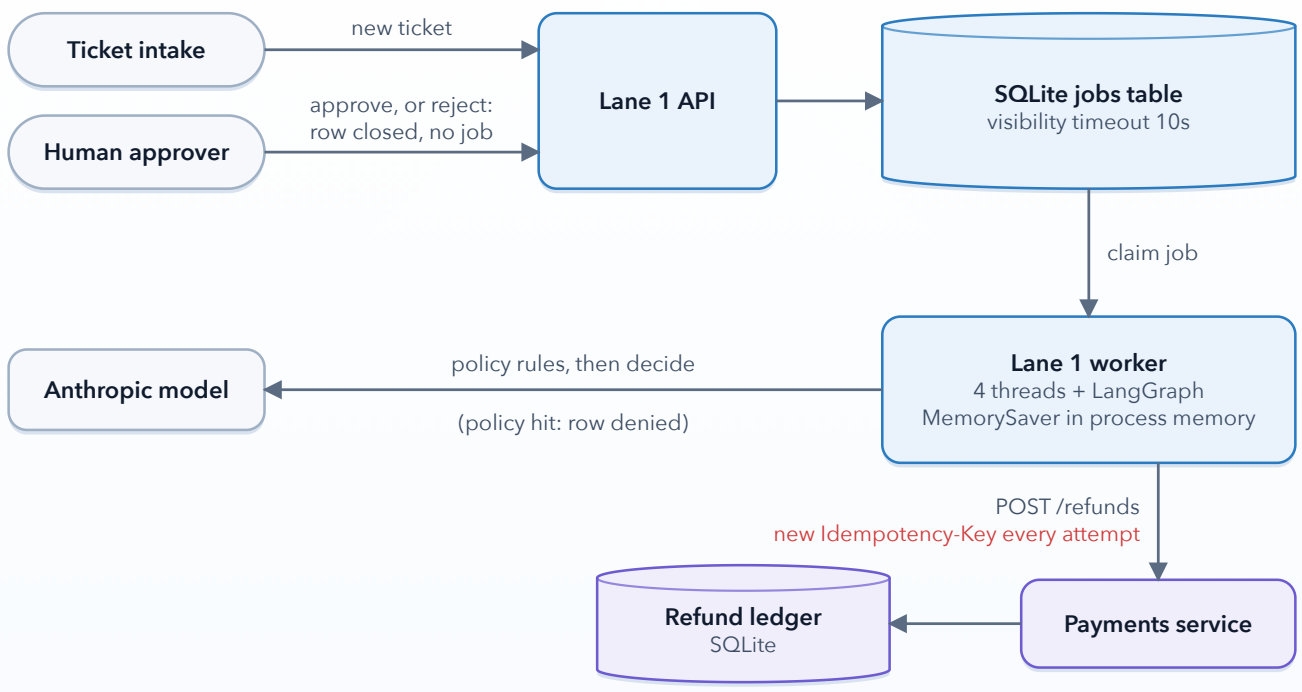


Figure 1 Today's refund path, as built in the repository ("Lane 1", shown on screen as Queue + LangGraph).

The company. An online marketplace with a support team, a payments platform team and a finance team. Support volume is high enough that an LLM agent drafts the first response to most tickets. The agent has just been extended from *drafting replies* to *proposing refunds*.

Users.

- Customers, who write in about a damaged item, a duplicate charge or a parcel that never arrived.
- Support agents, who approve refunds. An approval can take minutes, hours or days (a weekend, a manager out of office).
- Finance and risk, who must be able to answer "who approved this refund, and was it paid once?"

Integrations. A helpdesk that delivers tickets by webhook. An LLM provider. A payments API owned by the payments platform team, the only system allowed to move money.

Background jobs. A job queue with at-least-once delivery and a visibility timeout (the SQS or Pub/Sub shape), and worker processes that poll it. An approval arrives as an API call that enqueues an `execute` job. A `decide` job extends its lease while the model call runs, so only a dead worker's job is handed out again.

Who owns what.

Team	Owns
Support/AI	The agent, the ticket flow, the approval step, the worker fleet
Payments platform	The refund API, the ledger, the rules for what a refund request must carry
Finance/risk	The audit question and the loss number

How a refund works today.

1. A ticket arrives and a `decide` job runs the LangGraph graph. Policy rules in code go first (at most $200, order at most 30 days old); a ticket outside them is denied without calling the model. Otherwise the model proposes `refund` or `deny`.
2. For a refund, the graph pauses at an `interrupt` and waits for a human. The checkpointer is `MemorySaver`, the default in LangGraph's tutorials, so the pause lives in the worker's memory.
3. A human approves or denies. A denial closes the ticket. An approval enqueues an `execute` job, which resumes the graph and calls payments over HTTPS with a 3-second timeout, a few retries, and a **fresh idempotency key on every attempt**.
4. A recovery path re-drives tickets the database says are `executing`. An approved ticket whose in-memory graph is gone is marked `failed`.

This is deliberately what a competent team ships, not a strawman: it has a dedupe check for finished tickets, backoff, a lease heartbeat and a recovery path. Its two weak points are the common ones: the key is generated per attempt rather than per ticket, and the approval wait lives in process memory. Section 7 shows that every failure measured traces to one of those two choices.

02 Why it breaks

The failure that matters is *not knowing*. Payments committed the refund and the response never came back, or a worker died after the money moved. Retrying is then either unsafe (a new key means a second refund) or impossible (the in-memory approval is gone).

The long approval wait makes this worse. When an approval can wait hours or days, a deploy during the wait is routine, not rare ASSUMPTION, and every deploy restarts the workers that hold the pauses.

03 Who faces this, and why now

Companies like this one. Any company that moves money and has shipped an LLM agent that drafts support replies, and now wants it to act: refunds, credits, partial refunds, subscription changes. The telling structure is a payments platform team that owns the money-moving API and a separate support or AI team that owns the agent. Typical industries: e-commerce and marketplaces, fintech, travel and hospitality, subscription billing.
ASSUMPTION a profile, not a market study

Where the pain lands.

Role	The pain
Support engineering / AI platform	Agent runs stall or duplicate on deploys and crashes; long approvals are fragile; no clean way to prove what happened
Payments platform	Reluctant to hand an agent a raw refund API; wants a narrow, auditable contract and a contained blast radius
Finance / risk / compliance	A duplicate refund is a direct loss; needs proof that a human approved the exact action

Who champions it. The head of AI or support engineering, ideally with a payments staff engineer as co-champion. The payments team's buy-in is the real gate, because the Nexus contract is theirs to publish.

Who approves the investment. The VP of Engineering, with sign-off from payments and finance/risk, because money movement is involved. Expect pushback from the payments team (control of its API), security and compliance (the audit of approvals) and the platform team (one more system to run, whether Temporal Cloud or self-hosted).

Why now. Each of these is an assumption to check against your own situation:

1. Agents are moving from drafting to acting, and an approval gate is the obvious guardrail. That puts a human wait in the middle of an agent run.
2. Agent pilots tend to stall at production readiness, and the stall is in execution (state, retries, deploys), not in the model.
3. Something forces the issue: a duplicate refund, a lost approval, a ticket stuck after a deploy, or an auditor asking you to prove a human approved a payment.
4. Payments owns its API, so the agent team needs a contract, not direct access.

Precedent. Payments workflows are an established Temporal use. Mews, a hospitality platform, describes a Workflow that retries partial refunds on a schedule until the underlying transaction settles, and long-running chargeback Workflows driven by Signals from the payment provider's webhooks. DOCUMENTED `temporal.io/resources/case-studies/mews`, published by Temporal That supports "refund lifecycles fit Workflows and Signals". It says nothing about agents or Nexus, and I found no public source that does.

Not a fit. One team in one namespace, low volume, or no money movement. Nexus adds little there; child workflows and activities already compose work inside one team's namespace.

Questions to ask your own team.

- Does the agent take payment actions today, or is it advisory? What would it need before it could?
- Who owns the refund API, and how do they feel about an agent calling it?
- How long can an approval wait, and what happens to the run if you deploy during it?
- Have you had a duplicate or lost refund? How would you prove who approved one?

04 The options

A. Keep the agent advisory

The agent drafts; a human clicks refund in the payments console. This is the honest baseline and the right answer for a team that does not need the agent to act. It stops being right when the value of the agent is the action itself, or when the approval queue is the bottleneck.

B. Harden the current stack

The serious alternative. The repository measures the first fix; the rest are expected costs.

Fix	What it closes	Cost	Evidence
Derive the idempotency key from the ticket, not the attempt	Duplicates after commit-then-timeout, redelivery and crash-after-commit	One line. Use `refund-<ticket_id>` if you may migrate later (section 6)	**MEASURED** duplicates went from 9–10 per run to 0
A durable checkpointer (Postgres or similar) instead of `MemorySaver`	Approvals lost on restart or deploy	A checkpointer, a schema, a migration of in-flight state	**EXPECTED**
A recovery sweeper that re-drives orphaned `approved` tickets as well as `executing` ones	Tickets stranded by a crash	A scheduled job with its own failure modes, and care not to re-drive tickets already paid	**EXPECTED**
An outbox for the approval and the refund request	Messages lost between the API, the queue and the worker	Another table, a relay process, ordering rules	**EXPECTED**
A stuck-ticket dashboard	Visibility	Queries over your own tables	**EXPECTED**

With the key fixed, the duplicates went to zero, but the 3 approvals that were waiting when their worker died were still never paid **MEASURED**. The cheap fix removes the expensive failure and leaves the quiet one. Closing the quiet one needs rows two to four, which amounts to rebuilding a durable workflow engine one table at a time.

When hardening is the right answer. One team owns the whole path, approvals complete in seconds to minutes, volume is low, and the team already runs this kind of database-and-sweeper machinery. Make the key fix and revisit only if the approval wait grows.

When it breaks down. The approval wait is long, or payments belongs to another team that does not want to run the agent team's retry logic. Then you are building a durable timer, a state machine, a recovery sweeper and a cross-team contract, and testing all of them under crashes.

Hardened this way, Queue + LangGraph is a good system. Temporal + Nexus is better for this process because rows two to four are what the platform already does: the approval wait survives a restart because it is in the workflow's history, not because a sweeper finds it; retries follow one policy visible in one place; and the payments team owns its retries and key behind a contract. What Temporal does not remove is row one: the key still has to come from the ticket.

C. Another orchestration or durable-execution product

Cloud workflow services, queue-plus-state-machine setups and other engines can host the same process. I have not evaluated them against this scenario and make no measured comparison. The argument for Temporal here is narrower than "best engine": the approval wait maps to a signal with no timeout, the retry policy lives with the team that owns the call, and Nexus gives payments a typed contract across namespaces. Whether a competing product offers an equivalent cross-team contract is a question for your platform team. ASSUMPTION

D. Temporal without Nexus

An activity in the support workflow calls the payments API directly with the stable key. It works, it is simpler, and it is where to start if one team owns both sides. It leaves the agent team owning the payments client, timeouts and retry policy, and gives payments no place to enforce its own rules. Nexus costs a second namespace, an endpoint and a handler; it earns that only when the two sides belong to different teams.

The choice

Make B's key fix immediately in every case. Move refund approval onto Temporal when approvals wait long or a second team owns the money-moving API. Add Nexus when that second team exists. Keep A for anything that is not a money-moving action.

05 The migration: one process, refund approval

Why this process first. Not the whole agent and not every action it might take: the flow starts when the agent proposes a refund and ends when the money has moved once. A failure costs the most here (money), the boundary is clean (a ticket in, one refund out), and the outcome can be checked independently of the system that produced it (the payments ledger).

What stays as it is.

- The helpdesk webhook and ticket intake.
- The LLM provider and the agent's decision logic. The Temporal activity `run_agent` calls the same `agent.decide` function the existing graph calls, wrapped in a one-node LangGraph. OBSERVED one shared function in `paidonce/agent.py`
- The payments service and its API. Both paths call it over HTTPS with the same `Idempotency-Key` header; only the key's value differs. OBSERVED no change to the payments API was needed
- LangGraph, as the decision framework. What leaves is its in-memory pause and its retry loop around the payments call.

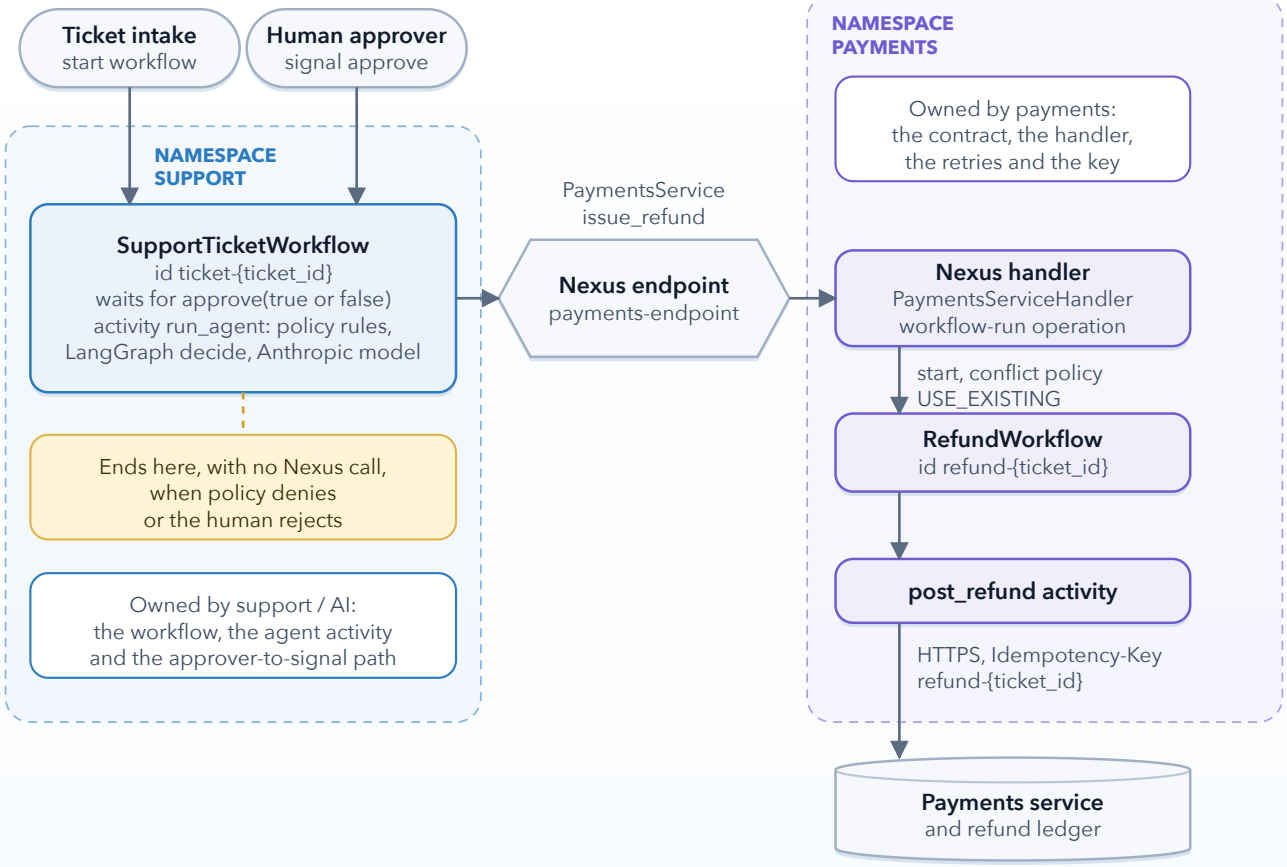


Figure 2 The same process on Temporal, across two namespaces: support in blue, payments in purple ("Lane 2", shown on screen as Temporal + Nexus).

What changes.

Concern	Today	On Temporal
Where a ticket's progress lives	A database row plus an in-process LangGraph checkpoint	The workflow's event history
The approval wait	An `interrupt` in worker memory	`wait_condition` on a signal, with no timeout
Approval delivery	An API call that enqueues an `execute` job	A signal to workflow `ticket-<ticket_id>`; a repeated signal is counted and ignored
Calling payments	An HTTPS call inside the graph, new key per attempt	A Nexus operation; payments' own workflow makes the call with key `refund-<ticket_id>`
Retries	An in-process loop, then a queue redelivery	An activity retry policy, owned by the team that owns the call

How big a change. The Temporal side of the demo is 382 lines of Python across nine small files, including both workers and the one-time setup MEASURED `wc -l paidonce/lane2/*.py`. That is the size of a demo, not an estimate: production adds the router, the approver-tool change, observability and deployment EXPECTED.

Alongside the existing services. Intake keeps working; a router (section 6) decides which path a new ticket takes; the approver tool sends a signal instead of enqueuing a job; payments and its ledger are untouched.

Nexus: one contract between two teams

The shape. The payments team publishes a contract, `PaymentsService.issue_refund`, which takes a `RefundRequest` (`ticket_id`, `amount_cents`) and returns a `RefundResult` (`refund_id`, `ticket_id`, `amount_cents`). The support workflow calls it through a Nexus endpoint, `payments-endpoint`. Behind the endpoint, payments runs a handler that starts a `RefundWorkflow`, whose activity calls the payments service. The support side imports the 16-line contract and its two data types and nothing else: it does not know the payments namespace, task queue, workflow type or retry policy. OBSERVED `paidonce/lane2/contract.py`

An early version also returned whether a retry had been answered from the ledger. That was payments' internal business, so it came out of the contract. A useful test for any Nexus contract: would the caller behave differently if it knew this field? If not, leave it out.

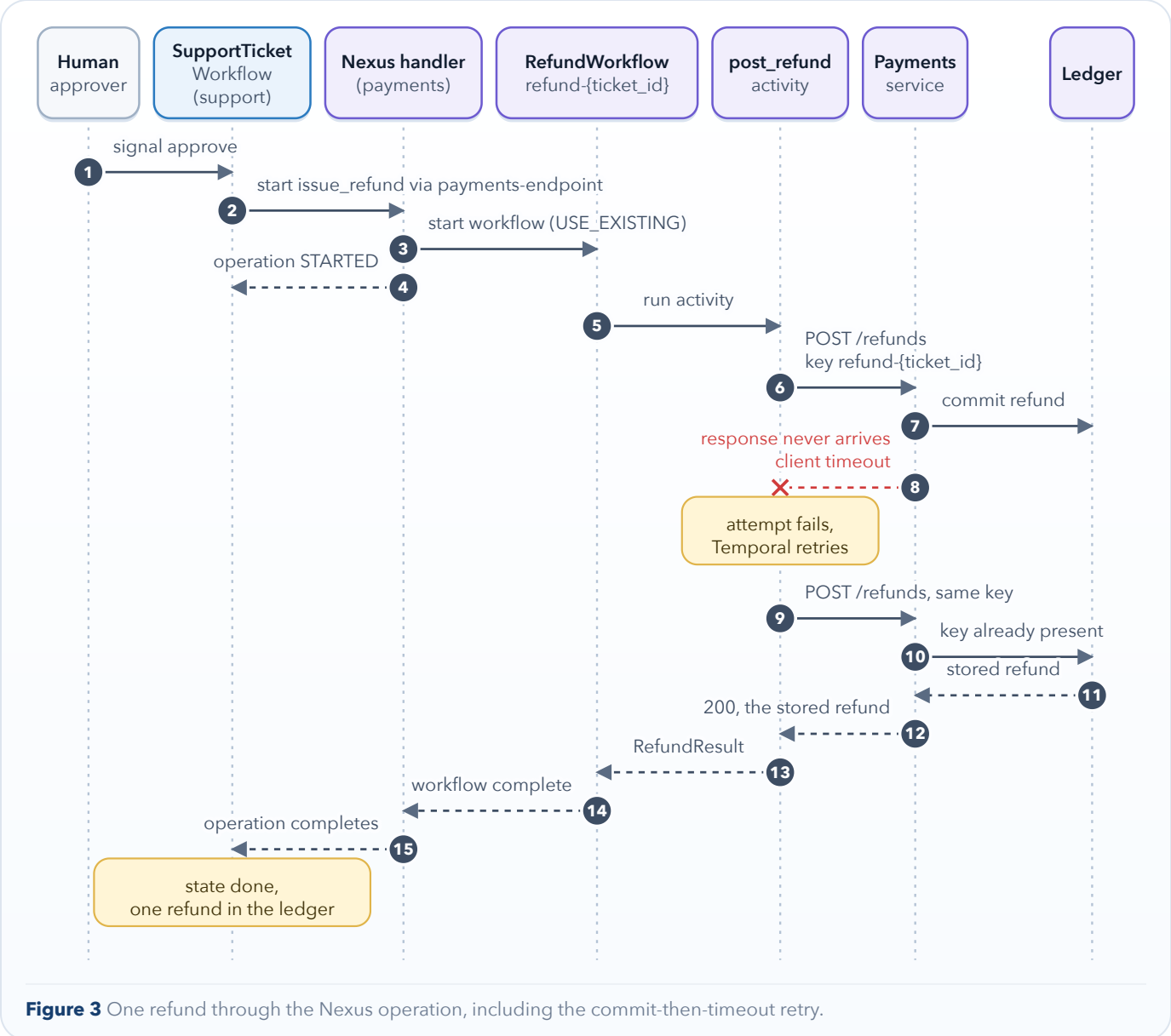


Figure 3 One refund through the Nexus operation, including the commit-then-timeout retry.

Why not call the payments API from an activity? For one team, that is the right answer. Across teams it leaves support holding payments' client, auth, timeouts and idempotency discipline, and payments cannot change how a refund is executed without a change on the caller's side. With Nexus, payments owns the execution and exposes an operation.

Delivery semantics, and why the design is what it is. Nexus operations execute at least once, and handlers should be idempotent, like activities. DOCUMENTED docs.temporal.io/nexus/operations The handler here is safe because the ledger refuses a second refund for the same key, and the key `refund-<ticket_id>` comes from the business operation. As a second layer, the handler starts its workflow with ID `refund-<ticket_id>` and the `USE_EXISTING` conflict policy, so a retried start attaches to the running workflow instead of starting another. DOCUMENTED docs.temporal.io/workflow-execution/workflowid-runid The ledger key is the guarantee; the workflow ID is the second layer.

Nexus features this work did not use (documented only, not built or tested):

- Asynchronous operations can run for up to 60 days, and cancelling a caller workflow propagates to its pending operations. DOCUMENTED docs.temporal.io/nexus/operations A denied or withdrawn approval cancelling a pending refund is a natural next step.
- An endpoint's target can be edited, and new operations route to the new target immediately. For a change of target namespace, completion callbacks stay with the original handler namespace, cancel requests go to the new target, and the docs recommend draining first. They do not describe a task-queue change with operations in flight. DOCUMENTED docs.temporal.io/nexus/registry
- Temporal Cloud gives each endpoint a caller-namespace allowlist; self-hosted deployments can implement custom authorizers. DOCUMENTED docs.temporal.io/nexus/security

06 Running the transition

The ledger audit exists in the repository (`paidonce/audit.py`); the router, the flag and the ownership split do not. Everything in this section is the plan EXPECTED until a team runs it.

Work already in progress.

- *Cut over at ticket creation, not mid-flight.* New tickets in the migrated cohort start on Temporal. Tickets already on the old path finish there, with a drain deadline.
- *A ticket never changes paths while it has an open refund.* If the two paths use different keys (the old path's `l1-<ticket_id>-<random>` , Temporal's `refund-<ticket_id>`), the ledger cannot recognise one path's refund when the other asks for the same ticket, because it enforces uniqueness on the key, not the ticket OBSERVED in `paidonce/payments/ledger.py` . A ticket the old path already paid and the new path then re-drives would be paid twice. The migration itself creates this hazard.
- *Close the hazard at the source.* Make the first-week key fix derive its key as `refund-<ticket_id>` , the same string the Temporal path uses. Then the ledger dedupes across both paths, in either direction, including after a rollback. (The measured fix used `l1-<ticket_id>` , the same mechanism with a different string; the shared-key variant was not run.)
- *Re-driving tickets the old path lost* is where the hazard bites. Look the ticket up in the ledger first and start a workflow only if no refund exists. Stronger still, payments adds a business rule: one refund per ticket unless the request is explicitly marked partial.

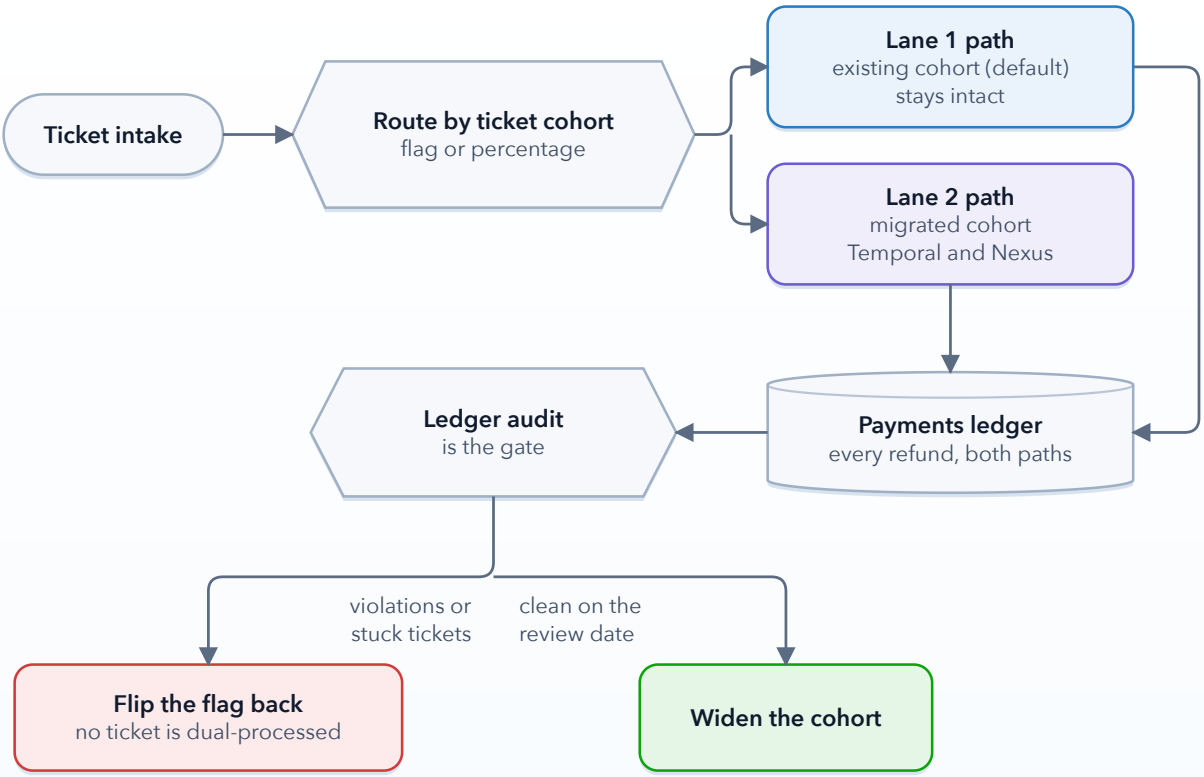


Figure 4 The migration shape: a router, a ledger gate and the rollback.

Operating both.

- Route by cohort, using a flag or a percentage of a hash of the ticket ID, so each ticket's path is stable.
- The ledger audit is the gate: a scheduled job that compares the approver tool's records with the payments ledger and never reads either path's own state. The repository's version is 152 lines.
- One dashboard per path for the duration, and one shared number: violations per cohort.

Rolling back.

- Flip the flag; new tickets go back to the old path, which stays intact throughout.
- Tickets already on Temporal are not stranded: their state is in Temporal, so they finish as long as the workers keep running. Rollback means "stop sending new work", not "kill the workers".
- Do not decommission the old path until the review date (section 9) has passed.

Ownership afterwards.

Team	Owns
Support/AI	`SupportTicketWorkflow`, the agent activity, the approver-to-signal path, the `support` namespace and its workers
Payments platform	The Nexus contract and endpoint, the handler, `RefundWorkflow`, the `payments` namespace and its workers, the ledger
Platform / SRE	The Temporal deployment (Cloud account or self-hosted cluster) and the shared runbooks

Each team is on call for its own namespace: a stuck refund workflow pages payments, a stuck ticket workflow pages support. The runbook must say what each does when a Nexus operation sits in `STARTED` for longer than expected.

07 Results

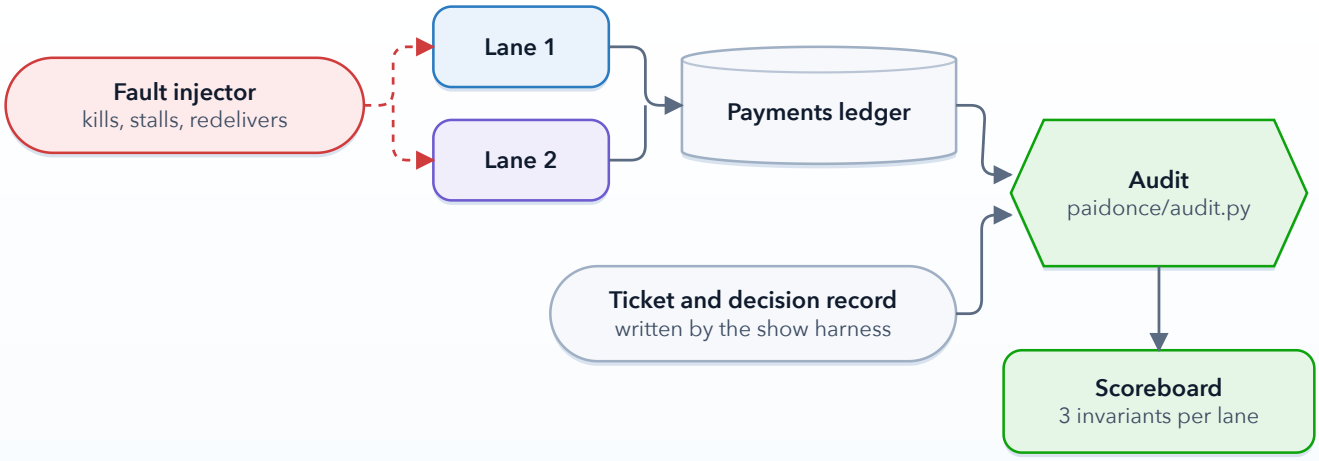


Figure 5 How both paths are scored: from the payments ledger and an independent record, never from either path's own state.

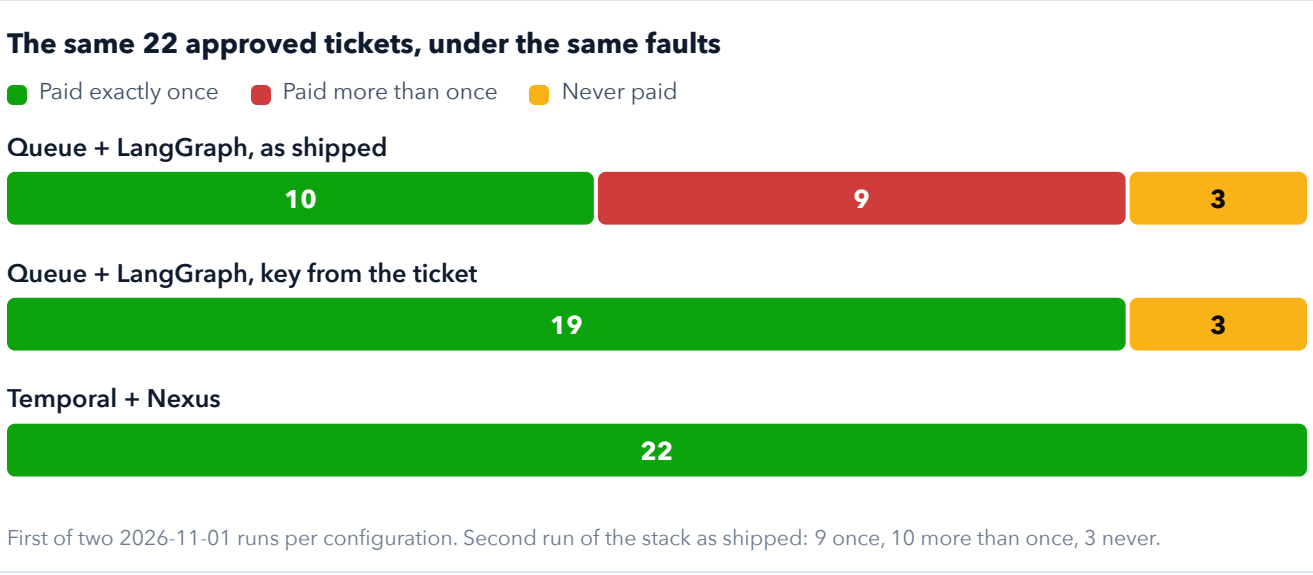
The claim under test. Moving refund approval onto Temporal, with Nexus to the payments team, keeps every approved refund exactly once and loses no ticket under worker crashes, queue redelivery, restarts during the approval wait and a payments service that commits and then stalls, without changing the agent logic or the payments API.

Method. `make scenario` sends the same tickets, approvals and faults through both paths at once. A fault injector kills workers, redelivers queue messages, restarts workers mid-wait, and makes payments commit and then hold its reply. Each fault scene's cohort has five approved tickets (six in the crash scene) plus two that must never be paid: one outside policy and one a human denies.

Scoring. A one-ticket walkthrough brings the approved total to 22 per path. An audit then reads the payments ledger and the harness's own record of what was submitted and decided, and checks three invariants per path: every approved ticket is refunded exactly once, no refund exists without an approval, and no ticket is lost or stuck. An approved ticket with no refund 20 seconds after its approval counts as lost. The command exits non-zero if the Temporal path breaks any invariant. Scored runs use a stub model, because the invariants depend on what is paid, not on the wording of the reply.

Results (2026-11-01, two runs of each configuration on one laptop; the same 22 approved tickets per path)

MEASURED



First of two 2026-11-01 runs per configuration. Second run of the stack as shipped: 9 once, 10 more than once, 3 never.

	Queue + LangGraph, as shipped	Queue + LangGraph, key from the ticket	Temporal + Nexus
Refunded exactly once	10 and 9	19 and 19	**22 and 22**
Paid more than once	9 and 10	0 and 0	**0**
Approved but never paid	3 and 3	3 and 3	**0**
Paid without an approval	0	0	0
Denied tickets paid (of 8)	0	0	0

By scene (one run each; duplicates / never paid for the existing stack; Temporal was 0 / 0 in every scene)

MEASURED

Scene	What it does	As shipped	Key from the ticket
Walkthrough	One ticket, no faults	0 / 0	0 / 0
Happy path	No faults	0 / 0	0 / 0
Crash or deploy	Kills the workers with 3 refunds in flight and 3 approvals waiting	3 / 3	0 / 3
Redelivery	After approval, the queue redelivers every refund job; on Temporal, three approval signals are sent again	3 / 0	0 / 0
Payments goes quiet	Payments commits, then holds its reply past the client timeout	4 / 0	0 / 0

The pattern matches the two weak points in section 1. The duplicates come from the per-attempt key; the never-paid tickets are approvals that were waiting in a worker's memory when it died. Earlier runs on a previous version of the scenes showed the same shape, and the Temporal path broke no invariant in any recorded run (seven with the default key, four with the stable key). Full tables are in `docs/scenario-results.md`.

The Nexus gauntlet OBSERVED one live-model run and one scored run. Five approved refunds in flight over Nexus, then four failures in a row:

1. Payments books each refund and holds its reply for 9 seconds.
2. The support worker is killed for about 5 seconds.
3. The payments worker is killed for about 6 seconds.
4. The Temporal server is killed for about 5 seconds and restarts from its database.

What each side saw:

- **Support:** the Nexus operation stayed `STARTED`, at attempt 1, with no error, the whole time. It would only see a failure after the operation's 10-minute schedule-to-close timeout. (Before payments accepts an operation, the caller retries the start itself; this holds once it has.)
- **Payments:** in the live run, each `RefundWorkflow`'s activity went through three attempts (the scored run had the same outcome). Attempt 1 gave up at the 3-second client timeout while payments held its reply; attempt 2 hit its 8-second start-to-close timeout when its worker was killed; attempt 3 succeeded.
- **The ledger:** each refund was booked once, inside attempt 1. Attempts 2 and 3 sent the same key, so payments answered them from the ledger: 15 calls for 5 refunds, 5 paid and 10 answered from the ledger. The audit scored all five as paid once, and the two denied tickets made no Nexus call at all.

Payments absorbed the stalled reply and its own worker's death with retries inside its namespace; the support-worker kill and the Temporal restart cost only time. I had expected the caller's operation to show the retries. For a workflow-run operation it does not, and that is the better result: containment.

Also observed.

- Intermediate retry attempts do not appear in a workflow's event history while the activity is retrying; they show only in its pending-activity state. The final attempt appears once it closes.
- A repeated approval signal is counted and ignored (the `duplicate_signals` count in the workflow's `status` query), and an integration test checks that duplicate signals never refund twice.
- With the live model, a separate run of two scenes had every eligible ticket reach the human with a refund proposal; Temporal refunded 10 of 10 once, and the existing stack paid 4 twice MEASURED one run.

What this evidence does not show.

- A rate. A handful of runs is a demonstration, not a sample, and the existing stack's counts vary by a ticket or two because kills land at wall-clock times. The reproducible claim is the Temporal outcome.
- Your system. The existing stack here is my model of a competent team's; the case for urgency rests on your own baseline (section 10).
- That durable execution alone made Temporal safe. The stable-key runs show the existing stack reaches zero duplicates with the same kind of key. Temporal's measured advantage over a hardened existing stack is the 3 approvals it does not lose, and the code you would write to stop losing them.

08 Benefits, costs and limitations

Benefits.

Benefit	Evidence
No ticket lost to a restart, including one during the approval wait	MEASURED 0 never-paid on Temporal; the existing stack lost the 3 approvals waiting through a restart
No duplicate refund after a crash, a redelivery or a payments timeout	MEASURED 0 on Temporal; the existing stack duplicated in all three fault scenes
A repeated approval does not act twice	MEASURED the redelivery scene, plus an integration test
A payments-side failure stays on the payments side	OBSERVED through a stalled reply and a payments-worker kill (plus a support-worker kill and a server restart), the caller saw `STARTED`, attempt 1, no error
Denied tickets are never paid	MEASURED 0 paid in every run on both paths
A clean contract between two teams	OBSERVED support shares one 16-line contract and two data types with payments

Costs (none measured):

- **Running Temporal.** A Temporal Cloud bill, or a cluster and its database to run, patch and monitor. The demo used a local dev server and says nothing about cost or effort. EXPECTED
- **A new discipline for workflow code.** Workflow code must be deterministic, and the Python SDK runs it in a sandbox that rejects heavy and filesystem-touching imports. That shaped the repository's layout: workflow modules apart from activity modules, activities called by name. Changing a workflow with executions in flight needs a versioning plan, which the demo does not exercise. OBSERVED for the layout EXPECTED for versioning
- **More moving parts.** Two namespaces, an endpoint, two worker fleets, and a runbook for an operation that sits in `STARTED`. EXPECTED
- **Effort beyond the demo.** The router, the approver change, observability and on-call, which I did not size. EXPECTED

The caveat that matters most: durable is not exactly-once. Temporal makes retries and restarts safe to *attempt*. The refund is exactly-once here because the key comes from the business operation and the ledger returns the stored refund when the same key comes back. A team that adopts Temporal but keeps a fresh key per attempt gets durable, diligent retries of a non-idempotent call, which is worse than before.

Limitations.

- Local processes and SQLite on one machine: this models crash and retry semantics, not scale, latency or network partitions.
- The approval waits were seconds long. That a days-long wait behaves the same is the documented behaviour of a workflow waiting on a signal, not something I ran.
- The audit scores the tickets and approvals the harness recorded. A production audit should also reconcile the other way: every ledger row has a recorded approval.

- The invariants say nothing about whether the refund *amount* was right, or whether the agent proposed the right thing.
- Not built: cancellation, the Cloud allowlist, a production router, endpoint retargeting with operations in flight.

09 Judging the pilot and deciding whether to expand

Set the review date when the migration starts, not when it ends; six weeks after the first migrated cohort goes live is a reasonable default. The thresholds below are proposals to agree between the teams. EXPECTED

Measure, per cohort (migrated versus not):

1. Ledger-audit violations: duplicate refunds, refunds without an approval, approved tickets with no refund.
2. Stuck tickets: approved more than N hours ago with no refund, with N taken from the existing path's approval-to-refund time.
3. Time from approval to refund, and the length of approval waits that survived a deploy.
4. Operating load: pages per week, time spent on workflow incidents, runbook steps used.

Decide:

At the review date	Decision
Zero duplicates and zero unapproved refunds in the migrated cohort; no stuck tickets past the threshold; approval-to-refund time no worse than baseline; operating load within what the teams agreed	**Expand.** Widen the cohort, then add the next action (credits, partial refunds)
Invariants clean, but operating load over the limit, or payments not yet comfortable owning the handler	**Hold.** Fix the operating model before widening
Any duplicate or unapproved refund in the migrated cohort that traces to the new design, or a stuck ticket the runbook cannot resolve	**Roll back** new tickets and find the cause before retrying

A violation that traces to something else (say, the agent proposing a wrong amount) is a finding, not a reason to roll back the platform.

10 A recommended path

Is this for you? The work pays off most when all three are true:

1. The agent takes, or is about to take, money-moving actions.
2. A human approval can wait hours or days, so an in-memory pause will not do.
3. A different team owns the money-moving API, so the agent team cannot simply fix the payments client.

If only the first is true, the key fix and Alternative B may be all you need.

In order.

1. **This week: the key fix, and a baseline.** Derive the key as `refund-<ticket_id>` (the string the Temporal path will use, so the dual-running hazard in section 6 never opens) and run a ledger audit over your existing path. A day or two of work gives you your own numbers for duplicates and lost tickets. EXPECTED the measured run used a different stable string
2. **First Temporal step: refund approval only,** inside one team's namespace, with an activity that calls payments using the stable key. No Nexus yet (Alternative D). Route a fixed cohort; gate on the ledger audit.
3. **Nexus when the second team is ready:** payments publishes `PaymentsService.issue_refund` and takes over the handler, the retry policy and the key.
4. **Expand by pattern:** other agent actions that need approval and cross a team boundary (credits, account changes), using the criteria in section 9.

What should change your mind.

- Your baseline shows few duplicates and no lost tickets: hardening is the right answer.
- The payments team will not publish a contract: stop at step 2.
- Running Temporal costs more than the failures it prevents at your volume: stop. ASSUMPTION needs your numbers

11 Lessons for teams building agent actions

1. **Key the side effect on the business operation.** `refund-<ticket_id>`, not a fresh UUID per attempt. Use the same string for the workflow ID. This single choice decided every duplicate in the study, on both stacks.
2. **Score from the system of record.** Record approvals independently, read the downstream ledger, run the same faults against old and new, and compare. It turns "trust us" into a number, and it doubles as the migration gate.
3. **Move the wait before you move the call.** The approval wait is where in-memory state dies. Putting it in a workflow is the part hardening cannot cheaply match.
4. **Start without Nexus; add it at the team boundary.** An activity is enough inside one team. Nexus pays off when another team should own the execution, the retries and the key.
5. **Know what the caller sees.** For a workflow-run Nexus operation, the caller's operation stays `STARTED` while the handler's workflow retries inside its own namespace. Alert on time-in-`STARTED`, not on caller errors, and look in the payments namespace for the retries.
6. **Look at pending-activity state for retries.** Intermediate retry attempts never appear in the event history; only the final attempt is recorded when the activity closes. A dashboard built on history alone misses them.
7. **Plan the module layout before porting the agent.** Keep workflow modules free of the LLM client, config loading and HTTP libraries; put them in activities. It is the first thing a team porting an agent trips on.
8. **Keep contracts small.** Ship only the fields the caller acts on. Internal facts, such as whether a retry hit the ledger, stay inside the owning team.

App. A Reproduce the results

The repository runs on macOS, Linux and Windows (WSL2). Install instructions are in the README. Then:

```
git clone https://github.com/agileresearchservices/paid-once.git && cd paid-once
make install
make scenario                              # both paths, stub model, about four minutes, no API key
PAIDONCE_L1_STABLE_KEY=1 make scenario     # the existing stack with the key from the ticket
make demo                                  # visual walkthrough with a live model (needs ANTHROPIC_API_KEY)
```

Design: `docs/architecture.md`. Full results: `docs/scenario-results.md`. Code tour: `docs/development.md`.

App. B Notes for Temporal's field, docs and product teams

Positioning. Lead with what hardening cannot give a customer. Duplicates are the loud failure, and a good team will fix them with a key change, so do not pitch Temporal as the cure for duplicate refunds: give the key lesson away. The pitch is *the approval wait and the cross-team call become durable and typed, and you stop writing the recovery code.* The line: *Queue + LangGraph is good once it is hardened; Temporal + Nexus is better because the hardening comes with the platform, and the team boundary comes with Nexus.* It concedes what is true about the existing stack and claims only what the evidence shows. A shorter draft: *Don't ask whether to trust the model. Make its actions durable, approved and auditable, and let the team that owns the money expose them through a contract it controls.*

Target customer. Narrower than "any team with an LLM agent": the three conditions in section 10. The stable-key runs show that a team with short approvals inside one team gets most of the way with a one-line fix.

Technical guidance worth publishing.

1. A short docs page on keying side effects from the business operation, with the per-attempt versus per-refund comparison.
2. A copyable Nexus skeleton for a cross-team side effect: a 16-line contract, a workflow-run handler that starts a workflow with a deterministic ID and `USE_EXISTING`.
3. A docs example showing both sides of one failure: the caller's operation in `STARTED` while the handler's activity retries.
4. The ledger-audit harness as a pattern for evaluations: same faults, old and new, scored from the system of record.

Sales conversations.

- Qualify with the questions in section 3, and walk away from the not-a-fit profile.
- *"Why not call payments from an activity?"* Fine for one team. Across teams it leaves the caller owning payments' client, auth and retry policy; Nexus lets payments own the execution.
- *"Why not harden what we have?"* You can, and the key fix closes real gaps. What hardening does not give you is a multi-day approval wait that survives a deploy without writing a durable checkpointer and sweeper yourself.

- Never promise exactly-once. Promise that retries and restarts are safe when the key comes from the business operation.

Friction log OBSERVED temporalio 1.35.0, nexus-rpc 1.4.0.

- I confirmed the Nexus and workflow-run handler signatures by reading the installed SDK source. An end-to-end Python example of a workflow-run operation with a conflict policy would have saved that.
- Intermediate retry attempts never reach the event history (only the final attempt does); while the activity retries, only its pending-activity state shows them.
- The sandbox's import rules forced the module layout; learnable, but the first thing a porting team hits.
- During the gauntlet, the pending activity's last failure for the held reply read "Failure exceeds size limit." instead of the timeout, and `current_retry_interval` returned an epoch-sized value on one poll, so the demo computes backoff from its own retry policy.
- Killing and restarting the dev server mid-scene with state intact requires starting it with a database file.
- The demo silently reuses any Temporal server already on the default port; it found another project's server once, and would have created namespaces and an endpoint in it. The README now warns about this.

App. C What my experience supports, and what needs validation

From my background: production agent workflows built with LangGraph; reliability work on search and ingestion systems, where retries, redelivery and partial failure are the normal case; and handling high-risk customer escalations as a technical account manager.

From the repository: the technical claims in sections 5 to 8, tagged MEASURED or OBSERVED.

Needs a real customer:

- Refund volume and the actual rates of duplicate and lost refunds; without a baseline, urgency is an assumption.
- Who owns approvals, and how long they wait in practice.
- Whether the payments team would publish a Nexus contract and run a handler.
- Whether the customer prefers to harden the current stack (section 4 has the argument and the one measurement).
- The real buying centre and who holds the platform budget.
- Cost and operating overhead of Temporal Cloud or self-hosting at their scale.
- Whether security and compliance accept the audit story, including how Nexus access is controlled (a caller-namespace allowlist per endpoint on Cloud, custom authorizers when self-hosted DOCUMENTED `docs.temporal.io/nexus/security`).